In [1]:
```
# Step 1: Download the dataset
import urllib.request

url = "https://raw.githubusercontent.com/SarahShafqat/Kaggle-Datasets/master/diabetes
urllib.request.urlretrieve(url, "diabetes.csv")

print("Dataset downloaded!")
```

```
Dataset downloaded!
```

In [2]:
```
# Step 1: Import required libraries
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall
```

In [3]:
```
# Step 2: Load the diabetes dataset
df = pd.read_csv("diabetes.csv")
print("Dataset shape:", df.shape)
print(df.head())
print(df.columns)
```

```
Dataset shape: (768, 9)
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0            6      148             72             35        0  33.6
1            1       85             66             29        0  26.6
2            8      183             64              0        0  23.3
3            1       89             66             23       94  28.1
4            0      137             40             35      168  43.1

   DiabetesPedigreeFunction  Age  Outcome
0                     0.627   50        1
1                     0.351   31        0
2                     0.672   32        1
3                     0.167   21        0
4                     2.288   33        1
Index(['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin',
       'BMI', 'DiabetesPedigreeFunction', 'Age', 'Outcome'],
      dtype='object')
```

In [4]:
```
# Step 3: Separate input features and target
target = "Outcome" if "Outcome" in df.columns else "diabetes"
X = df.drop(columns=[target])
y = df[target]
```

In [5]:
```
# Step 4: Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=1, stratify=y
)

# Step 5: Scale the features
scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

In [6]:
```
# Step 6: Train the KNN classifier
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train)
pred = knn.predict(X_test)
```

In [7]:
```
# Step 7: Evaluate the model
acc = accuracy_score(y_test, pred)

print("Confusion Matrix:\n", confusion_matrix(y_test, pred))
print("Accuracy:", acc)
print("Error Rate:", 1 - acc)
print("Precision:", precision_score(y_test, pred, zero_division=0))
print("Recall:", recall_score(y_test, pred, zero_division=0))
```

```
Confusion Matrix:
 [[85 15]
 [27 27]]
Accuracy: 0.7272727272727273
Error Rate: 0.2727272727272727
Precision: 0.6428571428571429
Recall: 0.5
```

In [8]:
```
# Step 8: Optimize k using 5-fold cross-validation
params = {"n_neighbors": np.arange(1, 26)}
grid = GridSearchCV(
    KNeighborsClassifier(), params, cv=5, scoring="accuracy"
)
grid.fit(X_train, y_train)

print("Best K:", grid.best_params_["n_neighbors"])
print("Best CV Accuracy:", grid.best_score_)
```

```
Best K: 19
Best CV Accuracy: 0.7605491136878583
```

In [9]:
```
# Step 9: Evaluate the best KNN model
best_knn = grid.best_estimator_
best_pred = best_knn.predict(X_test)
acc = accuracy_score(y_test, best_pred)

print("Confusion Matrix:\n", confusion_matrix(y_test, best_pred))
print("Accuracy:", acc)
print("Error Rate:", 1 - acc)
print("Precision:", precision_score(y_test, best_pred, zero_division=0))
print("Recall:", recall_score(y_test, best_pred, zero_division=0))
```

```
Confusion Matrix:
 [[89 11]
 [28 26]]
Accuracy: 0.7467532467532467
Error Rate: 0.2532467532467533
Precision: 0.7027027027027027
Recall: 0.48148148148148145
```

In [ ]: